Customer Setup Guide: Fast Portal Deployment (With or Without ARM)

This guide gives you **two fast, low-friction ways** to deploy in the Azure Portal by simply pasting JSON—no CLI/Bicep build pipelines required:

- **Fast Track (Portal ARM Paste — ~5 Minutes)**: Paste azuredeploy.json into **Deploy a custom template** in the Azure Portal to create the Analytics Rule, Logic App, Managed Identity, Sentinel Connection, and Automation Rule all at once, then run the 1-step permission grant in Phase 3.
- **Manual Portal Track (No ARM — ~10 Minutes)**: Follow Phases 1–4 below to create the Analytics Rule and Logic App in the Portal UI and paste portal-workflow-definition.json into the Logic App **Code view**.

Required Roles Checklist (Verify Before Starting)

Who	Required Role	Scope	Why It Is Needed
Deploying Admin (Azure)	`Owner` *(or* `Sentinel Contributor` + `Logic App Contributor` + `RBAC Administrator`*)*	Sentinel Resource Group	Creates the Analytics Rule, Logic App, Automation Rule, and assigns Azure RBAC roles.
Deploying Admin (Entra ID)	`Privileged Role Administrator` or `Global Administrator` *(Active in PIM)*	Entra ID Tenant	Grants `Machine.Isolate` and `Machine.ReadWrite.All` on `WindowsDefenderATP` (plus optional `Mail.Send` on `Microsoft Graph`) to the Logic App's Managed Identity. *(Note:* `Security Administrator` *and* `Azure Owner` *cannot grant Entra AppRoles).*
Deploying Admin (Exchange Online — Optional)	`Exchange Administrator` *(Only if scoping Graph* `Mail.Send`*)*	Exchange Online	Optional: Scopes the Logic App's `Mail.Send` permission to a single shared SOC mailbox via `New-ApplicationAccessPolicy` (or use the **Office 365 Outlook** connector in Designer with a mailbox sign-in instead).
Logic App Managed Identity *(Assigned in Phase 3)*	`Microsoft Sentinel Responder` + MDE `Machine.Isolate` & `Machine.ReadWrite.All` *(+ optional Graph* `Mail.Send`*)*	Resource Group + `WindowsDefenderATP` *(+* `Microsoft Graph`*)*	Reads Sentinel alerts, posts incident comments, isolates the endpoint in MDE, and sends optional email alerts.
`Azure Security Insights` *(Assigned in Phase 4)*	`Microsoft Sentinel Automation Contributor`	Resource Group	Allows the Sentinel Automation Rule to invoke the Logic App playbook.

> **Data Connector Prerequisite**: In **Microsoft Sentinel > Data connectors > Microsoft Defender XDR**, verify that **raw** `DeviceEvents` **table streaming** is checked (not just *Incidents & Alerts*).

Fast Track: Paste `azuredeploy.json` in Portal ("Deploy a custom template")

If your change-control allows pasting an ARM template inside the Azure Portal UI, you can skip Phases 1 and 2 completely:

1. In the Azure Portal search bar, open **Deploy a custom template** -> click **Build your own template in the editor**.
2. Paste the entire contents of azuredeploy.json and click **Save**.
3. Select your **Subscription** and **Resource group**, enter your existing **Workspace Name**, and click **Review + create** -> **Create**.
4. In the Logic App -> **Development Tools > API connections**, open the Sentinel connection -> **Edit API connection** -> **Save** to authorize the Managed Identity connection.
5. Skip directly to **Phase 3 (Grant Permissions)** below!

Manual Portal Track (No ARM): 4 Consolidated Phases

Before starting: approve this containment policy and confirm Logic Apps Consumption, the Sentinel connector and MDE API are available and authorized in your DoD/IL5 boundary. A matching USB vendor ID does not prove maliciousness. This workflow requests **full isolation**: third-party forensic tools will not retain network access. Test the MDE proxy/VPN connection and release path first.

Phase 1. Create the Sentinel Detection Rule & Copy Resource IDs

In your existing Sentinel workspace, go to **Analytics > Create > Scheduled query rule**. Use these settings:

Setting	Value
Name	`USB VID_03EB connected - automatic full isolation`
Severity	High
Run query every	5 minutes
Look up data from last	1 hour
Alert threshold	Greater than 0 results
Event grouping	Trigger an alert for each event
Suppression	Off
Create incidents	On
Group related alerts into incidents	Off
Rule enabled	**Off until the pilot**

Paste this query:

```
DeviceEvents
| where ActionType == "PnpDeviceConnected"
| where coalesce(ingestion_time(), TimeGenerated) > ago(5m)
| extend UsbFields = parse_json(AdditionalFields)
| extend USBDeviceId = tostring(UsbFields.DeviceId),
         USBDescription = tostring(UsbFields.DeviceDescription),
         USBClassId = tostring(UsbFields.ClassId),
         USBClassName = tostring(UsbFields.ClassName)
| where USBDeviceId startswith @"USB\VID_03EB"
| where isnotempty(DeviceId)
| summarize arg_max(TimeGenerated, *) by DeviceId
| extend MDEDeviceId = DeviceId, DetectionSignature = "USB-VID-03EB-v1"
| project TimeGenerated, DeviceName, MDEDeviceId, USBDeviceId,
          USBDescription, USBClassId, USBClassName, DetectionSignature
```

Under **Alert enrichment > Custom details**, add these exact keys:

Key	Query column
`MDEDeviceId`	`MDEDeviceId`
`DetectionSignature`	`DetectionSignature`
`USBDeviceId`	`USBDeviceId`

> **Why the `| extend MDEDeviceId = ...` line is required for the Custom Details dropdown:** In `DeviceEvents`, Microsoft uses the name `DeviceId` for two different things (`DeviceId` is the computer's 40-character MDE Machine ID, while `AdditionalFields.DeviceId` is the USB hardware ID). If you use your own custom KQL query instead of the one above, **you must include** `| extend MDEDeviceId = DeviceId, USBDeviceId = tostring(parse_json(AdditionalFields)["DeviceId"]), DetectionSignature = "USB-VID-03EB-v1"` at the bottom of your query—otherwise `MDEDeviceId`, `USBDeviceId`, and `DetectionSignature` will not appear in the **Custom details -> Value** dropdown!

Do not add a custom alert-title override. Save the rule. If the wizard creates it enabled, disable it immediately before connecting automation. Do not enable incident grouping later: this workflow intentionally processes only one matching alert/device per incident.

> **Flexible Scope Parameters (`workspaceResourceId` & `ruleResourceId`):** At the top of portal-code-view.json (lines 12 and 16), you can paste either the **plain GUID** (e.g., your Workspace ID GUID and Analytics Rule ID GUID) **or** the **full ARM Resource ID path** (`/subscriptions/.../workspaces/...`), or leave them blank (`""`) to allow any Sentinel incident in the workspace carrying `DetectionSignature = "USB-VID-03EB-v1"` and a 40-character `MDEDeviceId`.

Phase 2. Create the Logic App, Enable Managed Identity & Paste `Code view` (3 Easy Steps)

1. **Create the Logic App & System-Assigned Identity**:
   - Create a **Logic App (Consumption)** named `isolate-usb-03eb-dod` in the same resource group as your Sentinel workspace.
   - Open **Identity > System assigned > On > Save** and copy the **Object (principal) ID** for Phase 3.
2. **Add the Sentinel Trigger in Designer & Save**:
   - Open **Logic app designer** and add the **Microsoft Sentinel incident** trigger.
   - *Note on Auto-Connected Playbooks*: If another playbook already exists in your Resource Group, the Designer will automatically attach to that existing Microsoft Sentinel API connection. **You do NOT need to create a separate connection** as long as that connection uses `ManagedServiceIdentity`. If no connection exists yet, select **Authentication type: Logic Apps Managed Identity** and click **Create**.
   - Click **Save** on the Designer toolbar.
3. **Paste portal-code-view.json into `Code view` (3-Step Copy/Paste)**:
   - Open **Development Tools > Code view** in your Logic App.
   - **Step A**: Scroll to the very bottom of **Code view** (the last ~17 lines) and **copy** your `"parameters": { "$connections": { ... } }` block into Notepad so you don't lose your connection info:

```
    "parameters": {
      "$connections": {
        "value": {
          "azuresentinel": {
            "connectionId": "/subscriptions/.../resourceGroups/.../providers/Microsoft.Web/conn
ections/...",
            "connectionName": "...",
            "connectionProperties": {
              "authentication": {
                "type": "ManagedServiceIdentity"
              }
            },
            "id": "/subscriptions/.../providers/Microsoft.Web/locations/.../managedApis/azurese
ntinel"
          }
        }
      }
    }
```

- **Step B**: Select everything (`Ctrl+A`) in **Code view** and replace it by pasting the entire contents of portal-code-view.json.
- **Step C**: Scroll to the very bottom of **Code view**, replace the placeholder `"parameters": { "$connections": { ... } }` block with the block you copied in **Step A**, and click **Save**!
- *(Optional Email Alerts)*:
  - **Office 365 Outlook Connector in Designer (Easiest — No Graph Permissions Needed)**: After clicking **Save** in Code view, switch to **Logic app designer**, click inside `Process_incident`, and add **Office 365 Outlook -> Send an email (V2)** right after `Record_success`.
  - **Managed Identity Graph** `sendMail` **(No User Sign-in)**: At the top of **Code view**, set `notificationSenderEmail`'s `defaultValue` to your sender mailbox UPN (e.g., `"soc-alerts@agency.mil"`) and `notificationRecipientEmail`'s `defaultValue` to your recipient address (e.g., `"soc@agency.mil"`).

Phase 3. Grant Azure RBAC & MDE Permissions (One-Time Admin Step)

You can grant all required permissions in **one shot** using Azure Cloud Shell (recommended, no extra modules or `Connect-MgGraph` consent prompts required), or via the Portal IAM UI + `Microsoft.Graph`.

Option 3A: One-Shot Cloud Shell Commands (`az cli` + `az rest` — Fastest)

Run this in Azure Cloud Shell (PowerShell) as an active **Global Administrator** or **Privileged Role Administrator** who also has **Owner** / **RBAC Administrator** on the Resource Group:

```
$subId       = "PASTE-SUBSCRIPTION-ID"
$rgName      = "PASTE-RESOURCE-GROUP-NAME"
$principalId = "PASTE-LOGIC-APP-IDENTITY-OBJECT-ID"
$graphUrl    = "https://dod-graph.microsoft.us" # Use https://graph.microsoft.us (GCC High) or htt
  ps://graph.microsoft.com (GCC/Commercial)

# 1. Grant Microsoft Sentinel Responder to the Logic App Managed Identity
az role assignment create --assignee-object-id $principalId --assignee-principal-type ServicePrinc
  ipal `
    --role "Microsoft Sentinel Responder" --scope "/subscriptions/$subId/resourceGroups/$rgName"

# 2. Grant Microsoft Sentinel Automation Contributor to Azure Security Insights (so Automation Rul
  es can run the playbook)
$sentinelSpId = az ad sp list --filter "appId eq '98785600-1bb7-4fb9-b9fa-19afe2c8a360'" --query "
  [0].id" -o tsv
az role assignment create --assignee-object-id $sentinelSpId --assignee-principal-type ServicePrin
  cipal `
    --role "Microsoft Sentinel Automation Contributor" --scope "/subscriptions/$subId/resourceGrou
  ps/$rgName"

# 3. Auto-discover WindowsDefenderATP and grant Machine.Isolate + Machine.ReadWrite.All
$mdeSp = (az rest --method GET --url "$graphUrl/v1.0/servicePrincipals?`$filter=appId eq 'fc780465
  -2017-40d4-a0c5-307022471b92'" | ConvertFrom-Json).value[0]
foreach ($perm in @('Machine.Isolate', 'Machine.ReadWrite.All')) {
    $roleId = ($mdeSp.appRoles | Where-Object { $_.value -eq $perm -and $_.isEnabled }).id
    @{ principalId = $principalId; resourceId = $mdeSp.id; appRoleId = $roleId } | ConvertTo-Json
  -Compress |
        az rest --method POST --url "$graphUrl/v1.0/servicePrincipals/$principalId/appRoleAssignme
  nts" --headers "Content-Type=application/json" --body "@-"
}

# 4. (Optional - if using built-in Managed Identity Graph Email) Grant Microsoft Graph Mail.Send
$graphSp = (az rest --method GET --url "$graphUrl/v1.0/servicePrincipals?`$filter=appId eq '000000
  03-0000-0000-c000-000000000000'" | ConvertFrom-Json).value[0]
$mailRoleId = ($graphSp.appRoles | Where-Object { $_.value -eq 'Mail.Send' -and $_.isEnabled }).id
@{ principalId = $principalId; resourceId = $graphSp.id; appRoleId = $mailRoleId } | ConvertTo-Jso
  n -Compress |
    az rest --method POST --url "$graphUrl/v1.0/servicePrincipals/$principalId/appRoleAssignments"
   --headers "Content-Type=application/json" --body "@-"
```

Optional DoD IL5 Step: Scope Graph `Mail.Send` to a Single Shared SOC Mailbox (`O365USGovDoD`)

In DoD IL5 environments, security teams often require restricting application-level `Mail.Send` so the Logic App's Managed Identity can **only** send mail from a single designated shared/service mailbox (e.g., `soc-alerts@agency.mil`) rather than any mailbox in the tenant:

1. **Option 1 — Restrict Graph `Mail.Send` with an Exchange Online Application Access Policy (`O365USGovDoD`)**:

Run in PowerShell as an **Exchange Administrator** (using `ExchangeOnlineManagement` 3.x+):

```
   Connect-ExchangeOnline -ExchangeEnvironmentName O365USGovDoD

   # Create a mail-enabled security group containing only the allowed SOC sender mailbox(es)
   New-DistributionGroup -Name "Sentinel-Playbook-Allowed-Senders" `
       -PrimarySmtpAddress "sentinel-allowed-senders@agency.mil" -Type Security
   Add-DistributionGroupMember -Identity "Sentinel-Playbook-Allowed-Senders" `
       -Member "soc-alerts@agency.mil"

   # Restrict the Logic App Managed Identity (by its Application/Client ID) to ONLY that group
   $logicAppClientId = (az ad sp show --id $principalId --query appId -o tsv)
   New-ApplicationAccessPolicy -AppId $logicAppClientId `
       -PolicyScopeGroupId "sentinel-allowed-senders@agency.mil" `
       -AccessRight RestrictAccess `
       -Description "Restrict Sentinel USB Isolation Playbook to send mail only as soc-alerts@agen
  cy.mil"
```

2. **Option 2 — Use the `Office 365 Outlook` Connector in Logic App Designer (Zero Graph Permissions)**:
   - If your Entra ID / Exchange team prefers not to grant `Mail.Send` on Microsoft Graph at all, leave `notificationSenderEmail` and `notificationRecipientEmail` blank (`""`).

- After saving the Logic App, open **Logic app designer**, expand `Process_incident` -> `Send_success_email_if_configured` (or add an action right after `Record_success`), click **Add an action**, search for **Office 365 Outlook** -> **Send an email (V2)**, and sign in with the dedicated SOC service/shared mailbox account.

> **Fault-Tolerant Email Design**: If MDE Full Isolation succeeds (`Record_success`) but the email notification fails (for example, `Mail.Send` has not been granted yet in PIM or the sender UPN has no Exchange mailbox), the playbook catches the email error in `Record_email_warning`, posts a warning comment to the Sentinel incident, and still completes the Logic App run as `Succeeded` so MDE containment is never falsely reported as failed.

Option 3B: Portal IAM UI + `Microsoft.Graph` PowerShell

If you prefer assigning Azure RBAC in the Portal and using `Microsoft.Graph` on a workstation:

1. On the Log Analytics workspace (or Resource Group), go to **Access control (IAM) > Add role assignment**, select **Microsoft Sentinel Responder**, assign to **Managed identity**, and select your Logic App.
2. In PowerShell 7 with `Microsoft.Graph.Applications` installed (`Install-Module Microsoft.Graph.Applications -Scope CurrentUser`), locate the **WindowsDefenderATP** Enterprise Application Object ID and run:

```
$ErrorActionPreference = 'Stop'
$tenantId = 'PASTE-TENANT-ID'
$identityId = 'PASTE-LOGIC-APP-IDENTITY-OBJECT-ID'
$mdeApiObjectId = 'PASTE-MDE-API-ENTERPRISE-APP-OBJECT-ID'

Connect-MgGraph -Environment USGovDoD -TenantId $tenantId -ContextScope Process `
    -Scopes 'Application.Read.All', 'AppRoleAssignment.ReadWrite.All' -NoWelcome
$context = Get-MgContext
if ($context.Environment -ne 'USGovDoD' -or $context.TenantId -ne $tenantId) {
    throw 'Wrong Microsoft Graph cloud or tenant.'
}
$identity = Get-MgServicePrincipal -ServicePrincipalId $identityId -Property Id,ServicePrincipalTy
  pe
if ($identity.ServicePrincipalType -ne 'ManagedIdentity') {
    throw 'The supplied Logic App object ID is not a managed identity.'
}
$api = Get-MgServicePrincipal -ServicePrincipalId $mdeApiObjectId `
    -Property Id,AppRoles,ServicePrincipalNames
$audiences = @($api.ServicePrincipalNames | ForEach-Object { $_.TrimEnd('/') })
if ($audiences -notcontains 'https://api-gov.securitycenter.microsoft.us') {
    throw 'Wrong MDE API audience. Verify the government API registration before proceeding.'
}
foreach ($permission in @('Machine.Isolate', 'Machine.ReadWrite.All')) {
    $role = @($api.AppRoles | Where-Object {
        $_.Value -eq $permission -and $_.IsEnabled -and $_.AllowedMemberTypes -contains 'Applicati
  on'
    })
    if ($role.Count -ne 1) { throw "Expected one enabled application role: $permission" }
    $existing = @(Get-MgServicePrincipalAppRoleAssignment -ServicePrincipalId $identityId -All |
        Where-Object { $_.ResourceId -eq $api.Id -and $_.AppRoleId -eq $role[0].Id })
    if ($existing.Count -eq 0) {
        New-MgServicePrincipalAppRoleAssignment -ServicePrincipalId $identityId -BodyParameter @{
            principalId = $identityId
            resourceId = $api.Id
            appRoleId = $role[0].Id
        } | Out-Null
    }
    $verified = @(Get-MgServicePrincipalAppRoleAssignment -ServicePrincipalId $identityId -All |
        Where-Object { $_.ResourceId -eq $api.Id -and $_.AppRoleId -eq $role[0].Id })
    if ($verified.Count -ne 1) { throw "Permission not verified: $permission. Allow propagation an
  d rerun." }
    Write-Output "Verified: $permission"
}
```

`Machine.ReadWrite.All` is broad: MDE documents it for the machine and action-status

APIs used by the workflow. Customer approval is required. These MDE permissions

are separate from Sentinel workspace permissions.

Phase 4. Connect the Automation Rule & Test

1. **Grant Sentinel Playbook Permissions** *(skip if you already ran Option 3A above)*:
   - In **Microsoft Sentinel > Configuration > Settings > Settings > Playbook permissions > Configure permissions**, check the resource group containing your Logic App and click **Apply**.
2. **Create the Automation Rule**:
   - In **Microsoft Sentinel > Configuration > Automation > Create > Automation rule**:

Setting	Value
Name	`USB VID_03EB - full isolate`
Trigger	When incident is created
Condition	Analytics rule name Contains your rule created in Phase 1
Action	Run playbook -> select your Logic App (`isolate-usb-03eb-dod`)
Enabled	On, once ready for pilot/production

3. **Run an End-to-End Pilot or Simulated Test (3 Options)**:
   - **Option A — Zero-CLI Portal KQL Simulation (Recommended for Virtual Machines)**:
     1. In **Microsoft Sentinel > Logs**, find a non-critical test machine's 40-character `DeviceId` and `DeviceName`:

```
DeviceInfo
| where TimeGenerated > ago(24h) and isnotempty(DeviceId)
| summarize arg_max(TimeGenerated, DeviceName, OSPlatform) by DeviceId
| take 10
```

     2. Temporarily edit your Analytics Rule query and prepend this `SimulatedUsbEvent` block at the very top (replacing `<40-CHAR-MDE-DEVICE-ID>` and `<TEST-VM-HOSTNAME>`):

```
let SimulatedUsbEvent = print
    TimeGenerated = now(),
    DeviceId = "<40-CHAR-MDE-DEVICE-ID>",
    DeviceName = "<TEST-VM-HOSTNAME>",
    ActionType = "PnpDeviceConnected",
    AdditionalFields = todynamic('{"ClassId":"{36fc9e60-c465-11cf-8056-444553540000}","Cla
ssName":"USB","DeviceDescription":"Simulated Atmel USB Composite Device (VID_03EB Test)","Device
Id":"USB\\\\VID_03EB&PID_2404\\\\SIM03EB01"}');
union DeviceEvents, SimulatedUsbEvent
| where ActionType == "PnpDeviceConnected"
| extend AdditionalFields = todynamic(AdditionalFields)
| extend AdditionalFields_ClassId = tostring(AdditionalFields["ClassId"]),
        AdditionalFields_ClassName = tostring(AdditionalFields["ClassName"]),
        AdditionalFields_DeviceDescription = tostring(AdditionalFields["DeviceDescription
"]),
        AdditionalFields_DeviceId = tostring(AdditionalFields["DeviceId"])
| where AdditionalFields_DeviceId startswith "USB\\VID_03EB"
| project TimeGenerated, DeviceId, DeviceName, ActionType, AdditionalFields_ClassId, Addit
ionalFields_ClassName, AdditionalFields_DeviceDescription, AdditionalFields_DeviceId
| extend MDEDeviceId = DeviceId, USBDeviceId = tostring(AdditionalFields_DeviceId), Detect
ionSignature = "USB-VID-03EB-v1"
```

     3. Wait up to 5 minutes for the rule to fire, confirm the Sentinel Incident is created, the Logic App run succeeds, MDE Action Center reports `Isolate (Full) - Succeeded`, and the incident comment is written.
     4. Remove the `let SimulatedUsbEvent ...;` block, change `union DeviceEvents, SimulatedUsbEvent` back to `DeviceEvents`, save the rule, and release the test VM from isolation in MDE.
   - **Option B — Windows Guest VM PowerShell `SetupAPI` Simulation (`setupapi.dll`)**:

- On a test Windows VM (elevated PowerShell), you can register a software `USB\VID_03EB&PID_2404\SIM03EB01` PnP device node in Windows Device Manager (`HKLM\SYSTEM\CurrentControlSet\Enum\USB\VID_03EB&PID_2404\SIM03EB01`). See README.md — Method 2 for the full PowerShell snippet, a section-by-section explanation of each Win32 `setupapi.dll` call, and why MDE's kernel sensor (`MsSense.exe`) requires physical USB bus IRPs (or **Option A** above on Azure VMs) to emit a `DeviceEvents` row.

- **Option C — Physical USB Test or Automated Script (Simulate-UsbTestEvent.ps1)**:
  - Connect an approved test USB device matching `USB\VID_03EB` on a physical test endpoint, **or** run Simulate-UsbTestEvent.ps1 (with optional `-UnisolateAfterTest`) to automate the KQL injection, verification, and cleanup.
- Re-run the playbook on the same incident to verify duplicate-request prevention.

To stop automated containment at any time: disable the Automation Rule and Analytics Rule, then inspect queued workflow runs and pending MDE actions. Disabling rules does not automatically release already-isolated endpoints.

For limits, duplicate handling, logging considerations and Microsoft references,

see the operating notes.